PROBLEM 6 · SOLUTION

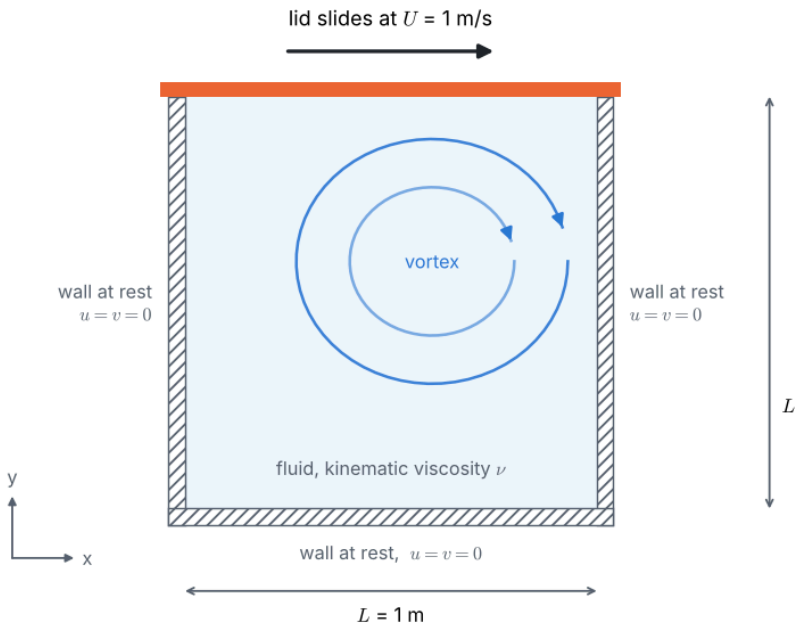
Lid-driven cavity flow

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math` and `time` from Python itself). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry and data

A square cavity of side $L = 1$ m is filled with a fluid of kinematic viscosity $\nu = 0.01$ m²/s. The lid slides to the right at $U = 1$ m/s; the other walls are at rest. Friction drags the fluid under the lid along, and a large vortex fills the cavity.



The cavity, the moving lid and the walls at rest.

The flow depends on one number only, the **Reynolds number** (inertia over viscous friction):

```
L = 1.0           # side of the cavity            [m]
U = 1.0           # speed of the lid              [m/s]
nu = 0.01         # kinematic viscosity           [m^2/s]

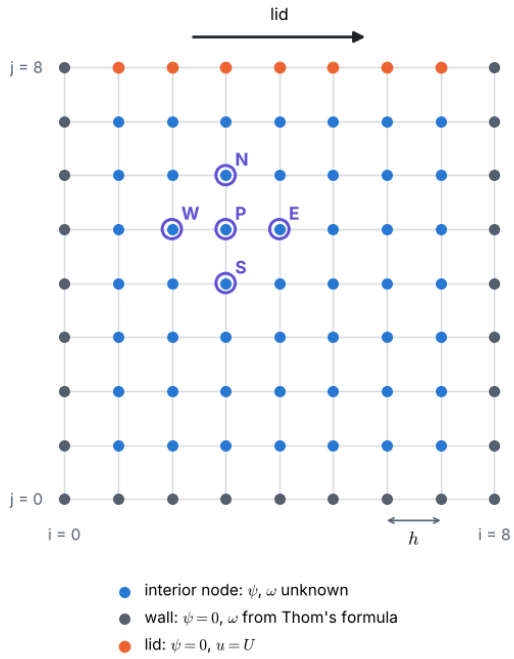
Re = U * L / nu
print(f"Reynolds number Re = U L / nu = {Re:g}")
```

```
Reynolds number Re = U L / nu = 100
```

Any cavity with $Re = 100$ has the same flow pattern, for example water ($\nu = 10^{-6}$ m²/s) in a 1 cm cavity with a lid moving at 1 cm/s.

Step 2 · The grid

A uniform grid with N intervals per side and spacing $h = L/N$; node (i, j) sits at $x_i = i\,h$, $y_j = j\,h$. Wall nodes are known; the $(N-1)^2$ interior nodes are unknown.



The grid (drawn with $N = 8$), with wall nodes, lid nodes and the five-point stencil of one interior node.

```
N = 32                    # number of intervals in each direction
h = L / N                 # grid spacing [m]

x = [i * h for i in range(N + 1)]    # node positions (the same list serves for y)

print(f"N = {N} intervals, h = {h:.5f} m, {(N + 1) ** 2} nodes, {(N - 1) ** 2} unknown nodes per
variable")
print(f"cell Reynolds number U h / nu = {U * h / nu:.3f}")
```

```
N = 32 intervals, h = 0.03125 m, 1089 nodes, 961 unknown nodes per variable
cell Reynolds number U h / nu = 3.125
```

The **cell Reynolds number** Uh/ν plays the role of the cell Péclet number of Problem 5.

Step 3 · Physics: from Navier–Stokes to stream function and vorticity

The steady, incompressible 2D flow obeys continuity and the two Navier–Stokes equations, three equations for u, v and p. Two definitions simplify them:

- **stream function** ψ: $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$. Then $\partial u/\partial x + \partial v/\partial y = \psi_{yx} - \psi_{xy} = 0$: continuity holds automatically. Lines of constant ψ are streamlines.
- **vorticity** $\omega = \partial v/\partial x - \partial u/\partial y$, the local spin of the fluid. Inserting u and v gives $\nabla^2\psi = -\omega$.

Differentiating the u-equation with respect to y and the v-equation with respect to x and subtracting removes the pressure and leaves the **vorticity transport equation**. The flow is described by two coupled

equations:

$$\underbrace{\nabla^2 \psi = -\omega}_{\text{Poisson, Problem 3}}, \qquad \underbrace{u\,\frac{\partial \omega}{\partial x} + v\,\frac{\partial \omega}{\partial y} = \nu\,\nabla^2 \omega.}_{\text{convection–diffusion, Problem5}}$$

Boundary conditions. No fluid crosses a wall, so every wall is a streamline: $\psi = 0$ on all four walls. The no-slip condition ($u = U$ on the lid, $u = v = 0$ elsewhere) gives the wall vorticity in Step 4.

Benchmark. There is no exact solution. We compare with the classic tables of Ghia, Ghia & Shin (1982), computed on 129 × 129 nodes:

```
# Ghia, Ghia & Shin (1982), Re = 100: u along the vertical centre line x = 0.5 ...
ghia_y = [1.0000, 0.9766, 0.9688, 0.9609, 0.9531, 0.8516, 0.7344, 0.6172, 0.5000,
          0.4531, 0.2813, 0.1719, 0.1016, 0.0703, 0.0625, 0.0547, 0.0000]
ghia_u = [1.00000, 0.84123, 0.78871, 0.73722, 0.68717, 0.23151, 0.00332, -0.13641, -0.20581,
          -0.21090, -0.15662, -0.10150, -0.06434, -0.04775, -0.04192, -0.03717, 0.00000]
# ... and v along the horizontal centre line y = 0.5
ghia_x = [1.0000, 0.9688, 0.9609, 0.9531, 0.9453, 0.9063, 0.8594, 0.8047, 0.5000,
          0.2344, 0.2266, 0.1563, 0.0938, 0.0781, 0.0703, 0.0625, 0.0000]
ghia_v = [0.00000, -0.05906, -0.07391, -0.08864, -0.10313, -0.16914, -0.22445, -0.24533, 0.05454,
          0.17527, 0.17507, 0.16077, 0.12317, 0.10890, 0.10091, 0.09233, 0.00000]
# ... and the primary vortex: minimum of psi and its position
ghia_psi_min, ghia_centre = -0.103423, (0.6172, 0.7344)

print(f"Ghia et al.: primary vortex psi = {ghia_psi_min} m^2/s at (x, y) = {ghia_centre} m")
```

```
Ghia et al.: primary vortex psi = -0.103423 m^2/s at (x, y) = (0.6172, 0.7344) m
```

Step 4 · Numerical scheme

Store ψ and ω as rows, `psi[j][i]`, `w[j][i]`. Interior node $P = (i, j)$, neighbours E, W, N, S.

Stream function (five-point formula, Problem 3):

$$\psi_P = \frac{\psi_E + \psi_W + \psi_N + \psi_S + h^2\,\omega_P}{4}.$$

Velocities (central differences): $u_P = (\psi_N - \psi_S)/(2h)$, $v_P = -(\psi_E - \psi_W)/(2h)$.

Vorticity (central differences for convection and diffusion, Problem 5 in 2D):

$$\omega_P = \frac{1}{4}\Big[\big(1 - \tfrac{R_x}{2}\big)\,\omega_E + \big(1 + \tfrac{R_x}{2}\big)\,\omega_W + \big(1 - \tfrac{R_y}{2}\big)\,\omega_N + \big(1 + \tfrac{R_y}{2}\big)\,\omega_S\Big],$$

$$R_x = \frac{u_P\,h}{\nu}, \qquad R_y = \frac{v_P\,h}{\nu}.$$

These are the CDS coefficients $a_W = D + F/2$, $a_E = D - F/2$ of Problem 5, in two directions, with the local cell Reynolds numbers in the role of Pe_Δ.

Wall vorticity: Thom's formula. On the lid, a Taylor series from the wall to the first interior node gives

$$\psi_{N-1} = \underbrace{\psi_{wall}}_{0} - h\,\underbrace{\frac{\partial \psi}{\partial y}\Big|_{wall}}_{U} + \frac{h^2}{2}\,\underbrace{\frac{\partial^2 \psi}{\partial y^2}\Big|_{wall}}_{-\omega_{wall}} - \dots \quad\Longrightarrow\quad \omega_{lid} = -\frac{2\,\psi_{N-1}}{h^2} - \frac{2U}{h},$$

because $v = 0$ along the whole wall makes $\partial v/\partial x = 0$, so $\omega_{wall} = -\partial u/\partial y = -\partial^2\psi/\partial y^2$. On the walls at rest the U term disappears: $\omega_{wall} = -2\psi_{adjacent}/h^2$. Corner nodes are never used.

```
def wall_vorticity(psi, w, N, h):
    """Thom's formula: vorticity on the four walls from the psi of the first interior nodes."""
    for k in range(N + 1):
        w[0][k] = -2 * psi[1][k] / h**2                  # bottom wall, at rest
        w[k][0] = -2 * psi[k][1] / h**2                  # left wall, at rest
        w[k][N] = -2 * psi[k][N - 1] / h**2              # right wall, at rest
        w[N][k] = -2 * psi[N - 1][k] / h**2 - 2 * U / h  # moving lid (top)
```

Step 5 · Solution setup: one iteration for two equations

Start from rest ($\psi = \omega = 0$) and repeat:

1. wall vorticity from the current ψ (Thom's formula);
2. one Gauss–Seidel sweep of the vorticity equation (velocities from the current ψ);
3. one Gauss–Seidel sweep of the Poisson equation for ψ (with the new ω);

until the largest changes of ω and ψ in one iteration are both below 10^{-6}.

```
def solve_cavity(N, Re, tol=1e-6, max_iter=100_000):
    """Steady lid-driven cavity: Gauss-Seidel on the vorticity and stream-function equations."""
    h = L / N
    nu = U * L / Re
    k = 1 / (4 * nu)                       # turns a difference of psi into half a cell Reynolds
number
    psi = [[0.0] * (N + 1) for _ in range(N + 1)]    # psi[j][i]: fluid at rest
    w = [[0.0] * (N + 1) for _ in range(N + 1)]      # w[j][i]
    changes = []                           # largest change of (w, psi) in every iteration

    for iteration in range(max_iter):
        # step 1: vorticity on the walls (Thom's formula)
        wall_vorticity(psi, w, N, h)

        # step 2: one Gauss-Seidel sweep of the vorticity transport equation
        largest_w = 0.0
        for j in range(1, N):
            for i in range(1, N):
                half_Rx = (psi[j + 1][i] - psi[j - 1][i]) * k     # u h / (2 nu)
                half_Ry = (psi[j][i - 1] - psi[j][i + 1]) * k     # v h / (2 nu)
                new = ((1 - half_Rx) * w[j][i + 1] + (1 + half_Rx) * w[j][i - 1]
                       + (1 - half_Ry) * w[j + 1][i] + (1 + half_Ry) * w[j - 1][i]) / 4
                largest_w = max(largest_w, abs(new - w[j][i]))
                w[j][i] = new

        # step 3: one Gauss-Seidel sweep of the Poisson equation for psi
        largest_psi = 0.0
        for j in range(1, N):
            for i in range(1, N):
                new = (psi[j][i + 1] + psi[j][i - 1] + psi[j + 1][i] + psi[j - 1][i]
                       + h * h * w[j][i]) / 4
                largest_psi = max(largest_psi, abs(new - psi[j][i]))
                psi[j][i] = new

        changes.append((largest_w, largest_psi))
        if largest_w < tol and largest_psi < tol:        # converged
            break

    return psi, w, changes
```

```python
def velocities(psi, N):
    """u and v at every node from central differences of psi (no-slip values on the walls)."""
    h = L / N
    u = [[0.0] * (N + 1) for _ in range(N + 1)]
    v = [[0.0] * (N + 1) for _ in range(N + 1)]
    for i in range(N + 1):
        u[N][i] = U                                    # the lid
    for j in range(1, N):
        for i in range(1, N):
            u[j][i] = (psi[j + 1][i] - psi[j - 1][i]) / (2 * h)
            v[j][i] = -(psi[j][i + 1] - psi[j][i - 1]) / (2 * h)
    return u, v
```

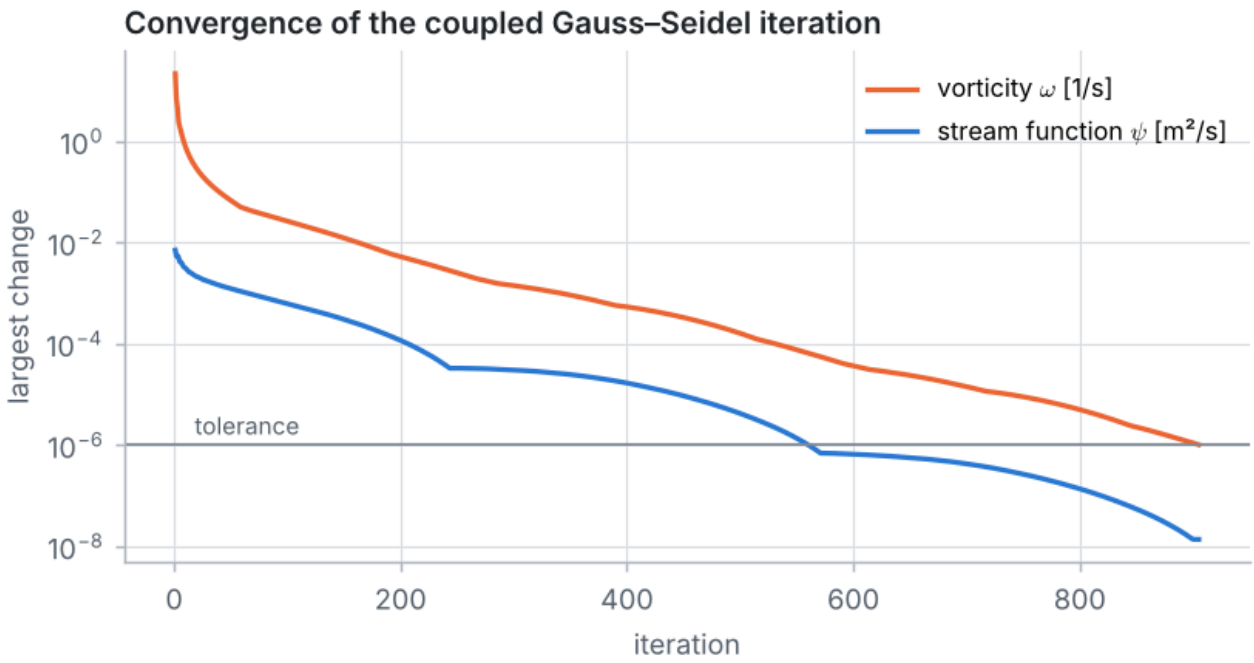
Why plain Gauss–Seidel? Over-relaxing the ψ sweep (SOR, Problem 3) roughly halves the number of iterations on fine grids, but in our tests it made the coarse grids ($N = 4$ and 8) diverge. Plain Gauss–Seidel converged on every grid from $N = 4$ to $N = 128$.

Step 6 · Compute

```python
start = time.time()
psi, w, changes = solve_cavity(N, Re)
seconds = time.time() - start

print(f"converged after {len(changes)} iterations ({seconds:.1f} s)")
print(f"last changes: vorticity {changes[-1][0]:.1e} 1/s, stream function {changes[-1][1]:.1e} m^2/s")
```

```
converged after 905 iterations (0.6 s)
last changes: vorticity 1.0e-06 1/s, stream function 1.3e-08 m^2/s
```



Largest change per iteration. The changes are largest while the flow spins up from rest, then fall steadily below the tolerance.

A small change per iteration is not the same as a small error (Problem 1). A run with a tolerance 100 times smaller checks it:

```
psi_tight, w_tight, changes_tight = solve_cavity(N, Re, tol=1e-8)
largest = max(abs(psi_tight[j][i] - psi[j][i]) for j in range(N + 1) for i in range(N + 1))
print(f"tol = 1e-8: {len(changes_tight)} iterations; psi changes by at most {largest:.1e} m^2/s")
```

```
tol = 1e-8: 1299 iterations; psi changes by at most 1.7e-06 m^2/s
```

The difference is more than a thousand times smaller than the effect of refining the grid (Step 7), so the tolerance is tight enough.

The whiteboard numbers. The whiteboard guide does the first iteration by hand on $N = 4$. Because the iteration starts from rest, `max_iter=1` returns the fields after one iteration:

```
def show(field, digits):
    """Print a field row by row, from the lid down (+ 0.0 turns -0.0 into 0.0)."""
    for j in range(len(field) - 1, -1, -1):
        print(f"   j = {j}: " + "  ".join(f"{value + 0.0:{digits + 4}.{digits}f}" for value in
field[j]))


for label, iterations in (("after iteration 1", 1), ("after iteration 2", 2), ("converged",
100_000)):
    psi_h, w_h, changes_h = solve_cavity(4, Re, max_iter=iterations)
    print(f"N = 4, {label} ({len(changes_h)} iterations)")
    print("  vorticity w [1/s]:")
    show(w_h, 4)
    print("  stream function psi [m^2/s]:")
    show(psi_h, 6)
```

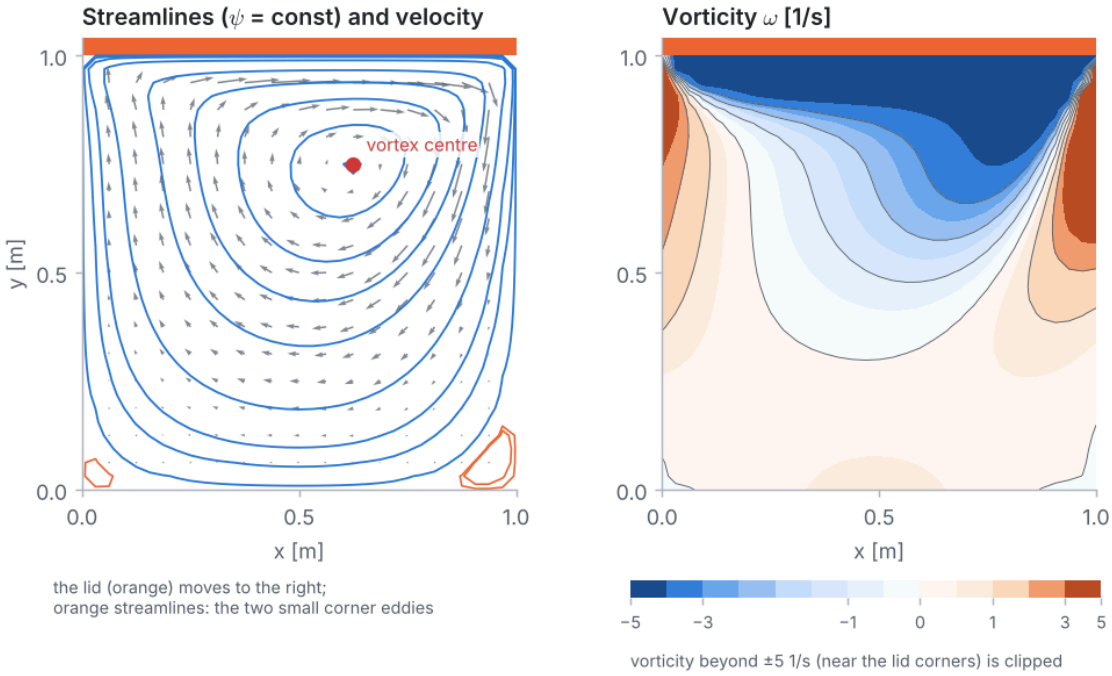
```
N = 4, after iteration 1 (1 iterations)
  vorticity w [1/s]:
   j = 4:   0.0000   -8.0000   -8.0000   -8.0000   -8.0000
   j = 3:   0.0000   -2.0000   -2.5000   -2.6250    0.0000
   j = 2:   0.0000    0.0000    0.0000    0.0000    0.0000
   j = 1:   0.0000    0.0000    0.0000    0.0000    0.0000
   j = 0:   0.0000    0.0000    0.0000    0.0000    0.0000
  stream function psi [m^2/s]:
   j = 4:   0.000000    0.000000    0.000000    0.000000    0.000000
   j = 3:   0.000000   -0.031250   -0.046875   -0.052734    0.000000
   j = 2:   0.000000    0.000000    0.000000    0.000000    0.000000
   j = 1:   0.000000    0.000000    0.000000    0.000000    0.000000
   j = 0:   0.000000    0.000000    0.000000    0.000000    0.000000
N = 4, after iteration 2 (2 iterations)
  vorticity w [1/s]:
   j = 4:   0.0000   -7.0000   -6.5000   -6.3125   -8.0000
   j = 3:   1.0000   -0.3457   -1.7268   -3.4112    1.6875
   j = 2:   0.0000   -0.5000   -0.6035   -0.6082    0.0000
   j = 1:   0.0000    0.0000    0.0000    0.0000    0.0000
   j = 0:   0.0000    0.0000    0.0000    0.0000    0.0000
  stream function psi [m^2/s]:
   j = 4:   0.000000    0.000000    0.000000    0.000000    0.000000
   j = 3:   0.000000   -0.021027   -0.051685   -0.073459    0.000000
   j = 2:   0.000000   -0.015625   -0.025055   -0.028951    0.000000
   j = 1:   0.000000    0.000000    0.000000    0.000000    0.000000
   j = 0:   0.000000    0.000000    0.000000    0.000000    0.000000
N = 4, converged (31 iterations)
  vorticity w [1/s]:
   j = 4:   0.0000   -7.4919   -7.0127   -6.0975   -8.0000
   j = 3:   0.5081   -0.3414   -0.3790   -2.8031    1.9025
   j = 2:   0.3624   -0.0185   -0.2390   -0.5387    1.0166
   j = 1:   0.1240    0.0722    0.0494    0.0356    0.3058
   j = 0:   0.0000    0.1240    0.2779    0.3058    0.0000
  stream function psi [m^2/s]:
   j = 4:   0.000000    0.000000    0.000000    0.000000    0.000000
   j = 3:   0.000000   -0.015879   -0.030854   -0.059454    0.000000
   j = 2:   0.000000   -0.011326   -0.024392   -0.031767    0.000000
   j = 1:   0.000000   -0.003875   -0.008683   -0.009556    0.000000
   j = 0:   0.000000    0.000000    0.000000    0.000000    0.000000
```

Step 7 · Post-processing and verification

```python
def vortex_centre(psi, N):
    """Node with the smallest psi: the centre of the primary vortex."""
    best = (0.0, 0, 0)
    for j in range(N + 1):
        for i in range(N + 1):
            if psi[j][i] < best[0]:
                best = (psi[j][i], i, j)
    value, i, j = best
    return value, i * L / N, j * L / N


psi_min, xc, yc = vortex_centre(psi, N)
print(f"primary vortex: psi = {psi_min:.5f} m^2/s at (x, y) = ({xc:.4f}, {yc:.4f}) m")
print(f"vorticity at the vortex centre: {w[round(yc * N)][round(xc * N)]:.4f} 1/s")
```

```
primary vortex: psi = -0.10038 m^2/s at (x, y) = (0.6250, 0.7500) m
vorticity at the vortex centre: -3.2091 1/s
```

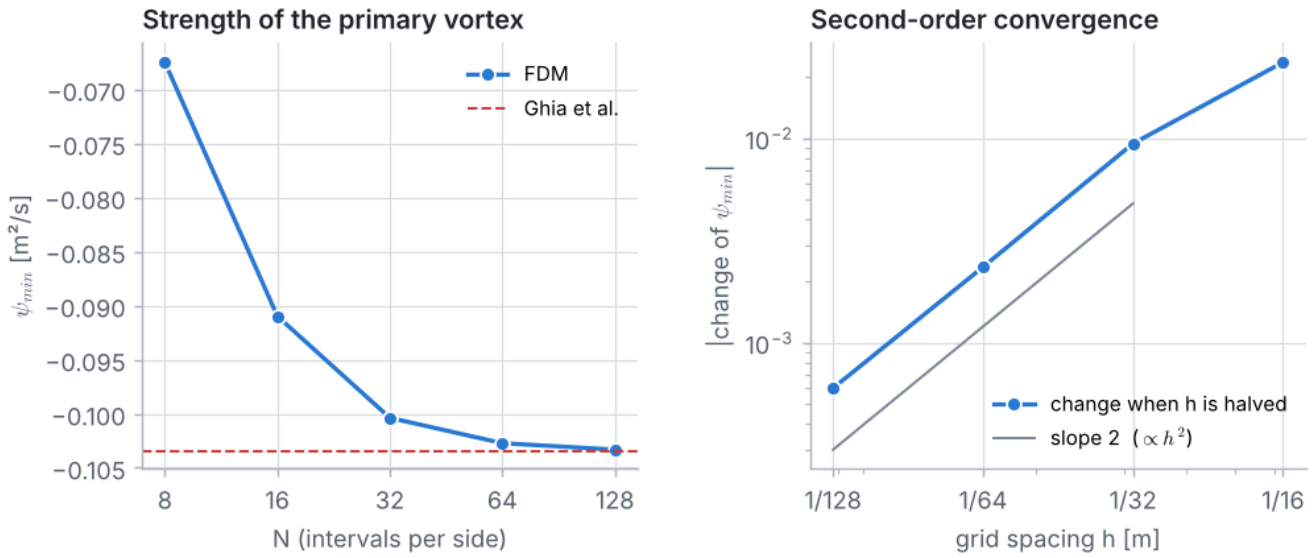


Left: streamlines and velocity arrows. One large clockwise vortex, its centre pushed up and to the right; two tiny anticlockwise eddies (orange) in the bottom corners. Right: vorticity, strongest in the thin layer under the lid where the velocity drops from U to almost zero.

Grid study. Solve the same flow on $N = 4$ to 128 (the finest grid has the same 129 × 129 nodes as Ghia et al. and takes about two minutes in plain Python):

```
grids = [4, 8, 16, 32, 64, 128]
results, psi_mins = {}, []
print("    N       h      iterations   time [s]   psi_min     change    ratio   centre (x, y)")
previous = None
for n_int in grids:
    start = time.time()
    psi_n, w_n, changes_n = solve_cavity(n_int, Re)
    seconds = time.time() - start
    value, xn, yn = vortex_centre(psi_n, n_int)
    results[n_int] = (psi_n, w_n)
    psi_mins.append(value)
    change = f"{value - previous[0]:+.6f}" if previous else "         "
    ratio = f"{previous[1] / (value - previous[0]):5.2f}" if previous and previous[1] else "     "
    print(f"{n_int:5d}   {L / n_int:.5f}   {len(changes_n):8d}   {seconds:8.1f}   {value:.6f}
{change}   {ratio}"
          f"   ({xn:.4f}, {yn:.4f})")
    previous = (value, value - previous[0] if previous else None)
print(f"Ghia et al. (129 x 129 nodes):                  {ghia_psi_min:.6f}
{ghia_centre}")
```

```
    N       h      iterations   time [s]   psi_min     change    ratio   centre (x, y)
    4   0.25000         31        0.0   -0.059454                       (0.7500, 0.7500)
    8   0.12500        136        0.0   -0.067492   -0.008038           (0.6250, 0.7500)
   16   0.06250        241        0.0   -0.090979   -0.023487    0.34   (0.6250, 0.7500)
   32   0.03125        905        0.5   -0.100382   -0.009403    2.50   (0.6250, 0.7500)
   64   0.01562       3297        7.6   -0.102725   -0.002343    4.01   (0.6094, 0.7344)
  128   0.00781      11932      119.8   -0.103324   -0.000599    3.91   (0.6172, 0.7344)
Ghia et al. (129 x 129 nodes):                  -0.103423                       (0.6172, 0.7344)
```

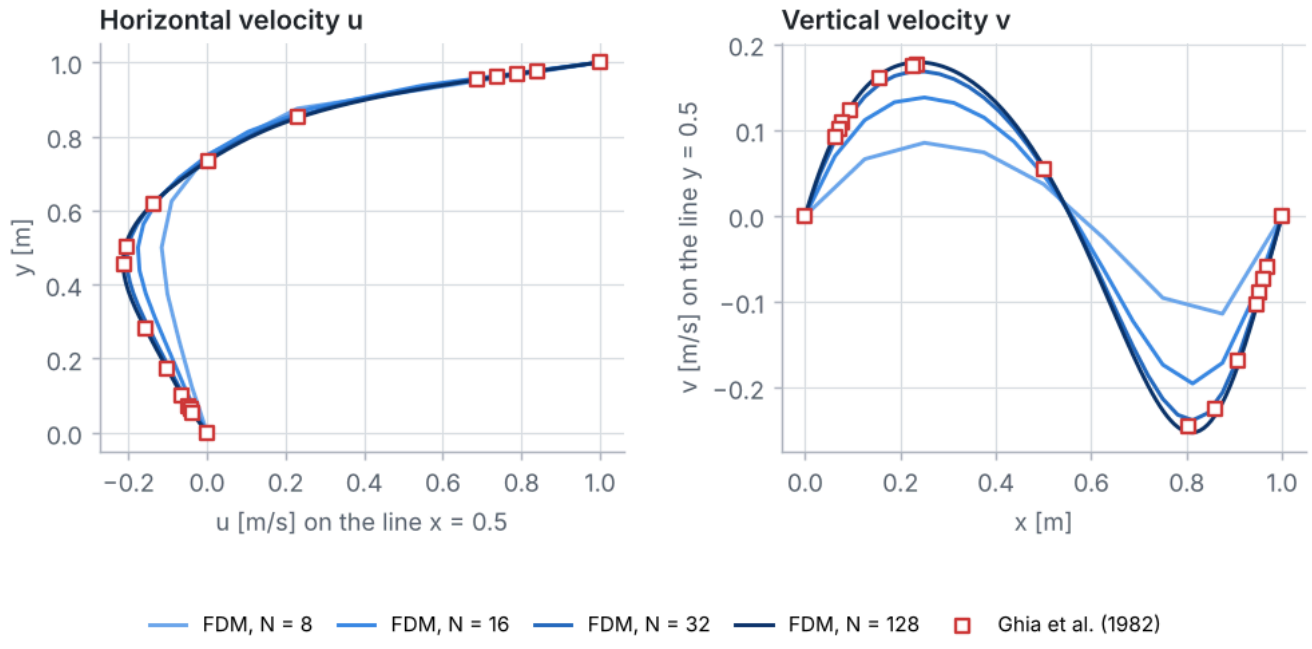


Left: ψ_{min} approaches the benchmark value as the grid is refined. Right: the change of ψ_{min} per halving of h falls along a line of slope 2.

- From $N = 32$ to 64 and from 64 to 128, the change of ψ_{min} falls by a factor of about 4: **second-order** convergence.
- On 129 × 129 nodes the vortex centre is at exactly Ghia's node, $(0.6172, 0.7344)$, and ψ_{min} agrees with Ghia's value to about 0.1 %.
- On $N = 4$ ($Uh/\nu = 25$) the vortex is far too weak: ψ_{min} is less than 60 % of the benchmark.

Centre-line velocities.

```python
def centre_lines(psi, N):
    """u along x = 0.5 and v along y = 0.5 (N must be even)."""
    u, v = velocities(psi, N)
    m = N // 2
    u_line = [u[j][m] for j in range(N + 1)]        # vertical centre line, bottom to top
    v_line = [v[m][i] for i in range(N + 1)]        # horizontal centre line, left to right
    return u_line, v_line
```



Velocity along the two centre lines on several grids, and the Ghia et al. benchmark (squares).

On 129 × 129 nodes the benchmark points fall on our nodes:

```
n_fine = 128
u_line, v_line = centre_lines(results[n_fine][0], n_fine)
print("    y      u (FDM)   u (Ghia)  difference  |     x      v (FDM)   v (Ghia)  difference")
worst_u = worst_v = 0.0
for k in range(len(ghia_y)):
    ju = round(ghia_y[k] * n_fine)                    # node closest to Ghia's point
    iv = round(ghia_x[k] * n_fine)
    du = u_line[ju] - ghia_u[k]
    dv = v_line[iv] - ghia_v[k]
    worst_u, worst_v = max(worst_u, abs(du)), max(worst_v, abs(dv))
    print(f"  {ghia_y[k]:.4f}  {u_line[ju]:8.5f}  {ghia_u[k]:8.5f}   {du:+.5f}   |
{ghia_x[k]:.4f}  "
          f"{v_line[iv]:8.5f}  {ghia_v[k]:8.5f}   {dv:+.5f}")
print(f"largest difference: u {worst_u:.5f} m/s, v {worst_v:.5f} m/s")
```

```
    y      u (FDM)   u (Ghia)  difference  |     x      v (FDM)   v (Ghia)  difference
  1.0000   1.00000   1.00000   +0.00000   |   1.0000   0.00000   0.00000   +0.00000
  0.9766   0.84313   0.84123   +0.00190   |   0.9688  -0.06208  -0.05906   -0.00302
  0.9688   0.79116   0.78871   +0.00245   |   0.9609  -0.07764  -0.07391   -0.00373
  0.9609   0.74017   0.73722   +0.00295   |   0.9531  -0.09302  -0.08864   -0.00438
  0.9531   0.69057   0.68717   +0.00340   |   0.9453  -0.10814  -0.10313   -0.00501
  0.8516   0.23573   0.23151   +0.00422   |   0.9063  -0.17653  -0.16914   -0.00739
  0.7344   0.00371   0.00332   +0.00039   |   0.8594  -0.23284  -0.22445   -0.00839
  0.6172  -0.13886  -0.13641   -0.00245   |   0.8047  -0.25255  -0.24533   -0.00722
  0.5000  -0.20870  -0.20581   -0.00289   |   0.5000   0.05736   0.05454   +0.00282
  0.4531  -0.21338  -0.21090   -0.00248   |   0.2344   0.17886   0.17527   +0.00359
  0.2813  -0.15701  -0.15662   -0.00039   |   0.2266   0.17865   0.17507   +0.00358
  0.1719  -0.10129  -0.10150   +0.00021   |   0.1563   0.16412   0.16077   +0.00335
  0.1016  -0.06413  -0.06434   +0.00021   |   0.0938   0.12583   0.12317   +0.00266
  0.0703  -0.04642  -0.04775   +0.00133   |   0.0781   0.11128   0.10890   +0.00238
  0.0625  -0.04179  -0.04192   +0.00013   |   0.0703   0.10314   0.10091   +0.00223
  0.0547  -0.03705  -0.03717   +0.00012   |   0.0625   0.09438   0.09233   +0.00205
  0.0000   0.00000   0.00000   +0.00000   |   0.0000   0.00000   0.00000   +0.00000
largest difference: u 0.00422 m/s, v 0.00839 m/s
```

Cell Reynolds number. The largest local value at an interior node:

```
print("    N    U h / nu    largest |u| h / nu or |v| h / nu at an interior node")
for n_int in grids:
    u_n, v_n = velocities(results[n_int][0], n_int)
    largest = max(max(abs(u_n[j][i]), abs(v_n[j][i])) for j in range(1, n_int) for i in range(1,
n_int))
    print(f"{n_int:5d}   {U * (L / n_int) / nu:7.3f}     {largest * (L / n_int) / nu:7.3f}")
```

```
    N    U h / nu    largest |u| h / nu or |v| h / nu at an interior node
    4    25.000       2.495
    8    12.500       3.375
   16     6.250       3.781
   32     3.125       2.523
   64     1.562       1.413
  128     0.781       0.744
```

It exceeds 2 on the grids up to $N = 32$, yet the iteration converges, and from $N = 32$ on the results converge at second order. The rule $Pe_\Delta \le 2$ of Problem 5 *guarantees* bounded results; breaking it in a thin layer under the lid does not necessarily spoil the solution. At higher Reynolds numbers this layer becomes thinner and the cell Reynolds numbers larger, so much finer grids or upwind-biased schemes become necessary.

Result ($Re = 100$). One large clockwise vortex fills the cavity, with two tiny counter-rotating eddies in the bottom corners. On the 33 × 33 grid the vortex centre is at $(0.625, 0.750)$ m with $\psi_{min} = -0.1004$ m²/s. Refining to 129 × 129 nodes gives $\psi_{min} = -0.10332$ m²/s at $(0.6172, 0.7344)$ m, within about 0.1 % of the Ghia et al. benchmark at the same position. The centre-line velocities agree with the benchmark to within 0.0042 m/s (u) and 0.0084 m/s (v), and the solution converges at second order.

Key ideas

- The **stream function** satisfies continuity automatically, and the **vorticity** removes the pressure: 2D Navier–Stokes = Poisson equation + convection–diffusion equation.
- The no-slip condition becomes a vorticity boundary condition through **Thom's formula**.
- Two coupled equations can be solved with one Gauss–Seidel iteration, starting from rest.
- Without an exact solution, a code is verified with a **grid study** and a published **benchmark**.